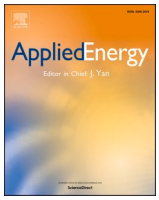
Applied Energy 393 (2025) 125971



Contents lists available at ScienceDirect

Applied Energy

journal homepage: www.elsevier.com/locate/apenergy





Coordinated scheduling optimization for Computility center microgrid considering computing resources dynamic pooling

Jian Zhao [a], Keran Huang [a], Yuan Gao [a,*], Xiaoyan Bian [a], Kai Zhang [b], Dongdong Li [a], Haoyang Cui [c]

[a] College of Electrical Engineering, Shanghai University of Electric Power, Shanghai 200090, China
[b] College of Computer Science and Technology, Shanghai University of Electric Power, Shanghai 200090, China
[c] College of Electronics and Information Engineering, Shanghai University of Electric Power, Shanghai 200090, China

HIGHLIGHTS

- A workload-core mapping model is proposed to evaluate the processors occupancy.
- A computing resources dynamic pooling method is proposed to integrate available cores based on the core real-time states.
- A self-regulating computility center microgrid coordinated framework is proposed to mitigate the volatility of PV generation.

ARTICLE INFO

Keywords:
Computility center
Operational state of processor cores
Computing resources dynamic pooling
Microgrid

ABSTRACT

The computility center (CC) is a flexible load that regulates its power demands through workload dispatching. CC microgrid can employ the flexibility of CC load to coordinate with the volatility of photovoltaic (PV) generation. However, the computing resources of CC are heavily fragmented. Such situation limits workload distribution and then results in CC load incapable of coordinating with microgrid. To address the above issue, this paper proposes a coordinated scheduling method for CC microgrid using computing resources dynamic pooling (CRDP). Specifically, a workload-core mapping model is proposed to transform workload into power load by formulating the matrix of processor core states. Subsequently, the CRDP method is proposed to integrate and allocate remaining available cores according to the core real-time states. Then a self-regulating CC microgrid coordinated framework is proposed to regulate the CC load by adjusting the scale of computing resource pool to the volatility of PV generation. The effectiveness of the proposed method in improving PV consumption is validated across different microgrid simulation scenarios.

1. Introduction

Computility center (CC) is the infrastructure of high-performance computing systems [1]. As societal demand for data computation escalates, the quantity of CC increases rapidly, resulting in the growing proportion of CC load in total societal electricity consumption [2]. Meanwhile, the CC is high-quality flexible load that valuable participants in microgrid scheduling [3,4]. By altering the workload processing times, the CC can regulate power demand due to certain workloads characteristics of the time-delayed processing [5]. Then CC can allocate workloads across suitable servers within seconds based on workload requirements in the microgrid scheduling [6].

At present, the optimization of microgrid considering the flexibility of CC load scheduling is still in the preliminary stages of research. Li et al. [7] leverage the flexibility of loads in CC to optimize the operational cost of the microgrid, including operation cost, power purchase cost, and workload shedding cost. Demand response capability of the internet data center microgrid is achieved through the proposed optimal solution. Qi et al. [8] increase the flexible ratio of workloads and decrease the PUE to reduce the total planning cost in microgrids. The proposed integrated planning problem studies and models both the long-term investment and short-term operational details of distributed data center microgrids. Wan et al. [9] develop an energy transition framework considering the dispatch of CC to smooth the energy transition

* Corresponding author.
E-mail address: gaoyuan@shiep.edu.cn (Y. Gao).

https://doi.org/10.1016/j.apenergy.2025.125971
Received 6 December 2024; Received in revised form 6 March 2025; Accepted 18 April 2025
Available online 12 May 2025
0306-2619/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

process with the lowest load. Liu et al. [10] propose a proactive resilient unit commitment with CC in the day-ahead market to enhance the resilience of power system. The scheduling process is modeled as a two-stage distributionally robust optimization problem. Chen et al. [11] employ an incentive-compatible CC demand response strategy to lower the power system operation cost. An operation scheduling optimization model is proposed to minimize the operating costs of integrated energy system in [12]. Lian et al. [13] propose a robust multi-objective optimization model based on an adjustable policy under a flexible uncertainty set. The model simultaneously optimizes wind power curtailment, operational cost, and surplus computation resources, while accounting for uncertainties in wind power and real-time workload. Cao et al. [14] explore a data-driven flexibility assessment scheme of internet data center by investigating the temporal shifting capability of periodic batch workloads. They develop a four-step assessment procedure using a data-driven approach. Ding et al. [15] design an integrated energy management scheme to optimize the operation cost for a data center microgrid. Considering the uncertain nature of renewable supply, as well as power and heat demand, a two-stage stochastic programming model is formulated. In microgrids with high penetration of distributed photovoltaic (PV) generation, the key to mitigating the variability of PV generation lies in ensuring that the power generation and load within the microgrid remain balanced at all times. The CC plays a critical role in maintaining this equilibrium by acting as a flexible load. Microgrid can function both as a load and a generation source due to its integrated energy storage, enabling it to stabilize PV generation fluctuations. For both systems, a major challenge in achieving power balance is leveraging the full flexibility of CC load scheduling to effectively counteract PV variability. Specifically, the challenge lies in delving further to model CC loads with greater precision, while simultaneously enhancing the clarity of their scheduling flexibility. Nevertheless, previous researches have only focused on utilizing the flexible workloads within the CC, neglecting this challenge. In practical, workloads are executed directly on processor cores within CC. The existing researches only model CC loads at the server level, lacking the necessary granularity to address resource fragmentation and load scheduling flexibility in CC. In practice, the CC commonly exhibits phenomena such as resource expansion limitation, resource heterogeneity, and resource fragmentation. Among these situations, resource fragmentation is particularly prominent, leading to an insufficient scale of idle computing resources to meet application requirements. Hence, thorough consideration of the collaborative between flexible workloads and computing resources in CC microgrid is urgently needed to optimization scheduling.

However, the above researches have overlooked the impact of the widely fragmented computing resources on the coordinated scheduling of CC microgrids, which limits the scheduling potential. Specifically, the CC workload exhibits heterogeneity. This heterogeneity is reflected in workloads requiring a varying number of processor cores across different time periods. The remaining available processor resources of the server are fragmented due to the processing of heterogeneous workloads in CC. When the quantity of fragmented resources is lower than the workload processing requirements, workload shifting across time is difficult to achieve. Such situation limits the load scheduling capacity of CC as flexible loads. In this situation, CC load cannot be used to coordinate with PV generation effectively. This is primarily due to the mismatch between computing resources and workloads. For example, fixed ratios of central processing unit (CPU) computility and graphics processing unit (GPU) computility on a single server cannot accommodate the diverse demands of workloads. Zeng et al. [16] provide a workload computation strategy based on task requirements and processor state assignment to optimize overall performance and efficiency. This work achieves energy efficiency through the design of joint resource management for computing and communications. Rosenfeld et al. [17] devise a heterogeneous query processing system to solve the problem of distributing workloads on processors with different capabilities, which can overcome the data transfer bottleneck and support implementations for multiple processors efficiently. Ye et al. [18] describe how existing schedulers manage their respective workloads to achieve scheduling goals and optimize resource utilization for the efficient use of computational resources. Besides, workloads are unable to be shifted due to remaining computing resources on the allocated servers, which fail to satisfy the requirements of workloads. Weak workload scheduling ability of CC can lead to insufficient allocation of computing resources. Demoulin et al. [19] design a new scheduling policy, dynamic application-aware reserved cores, to dynamically profile the workload. The purpose of this scheduler is to minimize application latency. Iqbal et al. [20] propose a deep learning-based adaptive window size selection method to capture the trend for the latest resource utilization. Any estimation algorithm can use this window to train a model to predict the observations in the next interval. Zhang et al. [21] develop an optimization model considering workload distribution and computational resource allocation in edge computing, which can deal with stochastic workloads. Unlike common scheduling strategies for microgrids, the characteristics of computing resources must be considered when the CC participates in power dispatch, such as fragmentation. The lack of consideration for the relationship between these two sides can lead to lead to lower utilization rates. Such situations will cause additional challenges for computility center to capture the flexibility of workload regulation.

Several works on computing resource allocation could be applied to deal with the fragmentation. This study provides some favorable references for the coordination of CC microgrid. These works can be generally divided into two categories. The first strategy focuses on the reasonably allocation of workloads to improve resource utilization [22–30]. Liu et al. [22] establish a job scheduling model considering job dependency and heterogeneity to obtain online scheduling policy for low-carbon data center operation. Kang et al. [23] propose an online scheduling algorithm to optimize job scheduling decisions on the premise that the job execution time cannot be accurately known. Zhou et al. [24] apply a collaborative scheduling strategy to mitigate the impact of proactive internet data center workload transfers on power system balance. A job scheduling scheme was designed within the data center in association with the feature set to reduce resource wastage in [25,26]. Considering workloads importance, references [27, 28] address a dynamic single machine scheduling problem with predicted arriving jobs. Fan et al. [29] propose a resource scheduling strategy that dynamically and efficiently searches for computing resources and allocation. The second category utilizes virtual machine consolidation technology to reduce resource fragmentation [30,31]. Ma et al. [32] develop an online virtual machine scheduling scheme for joint energy consumption and cost optimization that divides the scheduling process into two parts: virtual machine allocation and virtual machine migration. Ye et al. [33] propose a mission-driven two-step virtual machine commitment by considering the characteristics of UPS power losses in virtual machine allocation. Abohamama et al. [34] employ multi-dimensional resource-aware allocation policies to balance the use of different resources (CPU, memory and bandwidth) in each server to improve energy consumption and resource wastage in cloud environments. Zhou et al. [35] develop a novel adaptive energy-aware virtual machine allocation and deployment mechanism to efficiently handle the fluctuation of load. Liu et al. [36,37] propose a multi-objective optimization model and reasonably allocating virtual machine in as few physical machines as possible can reduce energy consumption. Maldonado-Carrascosa et al. [38] explore a game theory-based virtual machine migration method that dynamically reallocates virtual workloads to better utilize resources and optimize energy consumption. However, the flexibility of workloads remains largely unexplored in terms of deeply coordinated computing resources fragmentation and PV generation. The existing workload distribution approaches are unable to overcome server constraints in workload dispatching and computing resource distribution due to the fixed number of processor cores in a

single server. Most existing studies on virtual machine consolidation fail to accommodate the dynamic fluctuations of user workloads and varying load requirements. Furthermore, this approach employs a coarse-grained allocation of computing resources, resulting in inadequate fine-grained management capabilities. Therefore, existing approaches fail to fully realize the scheduling potential of CC loads. While it needs further research on how to coordinate the flexible load adjustment abilities of CC to mitigate fluctuations of PV generation.

Based on the literature review and the above discussion, it can be concluded that there is a lack of a coordinated scheduling method that considers both computing resources and workloads, effectively describing how computility center can participate in microgrid scheduling. Although existing literature has addressed flexible workloads, few studies comprehensively consider both fragmentation and flexibility simultaneously within CC microgrid optimization. Optimizing for workload and power load often requires different technical approaches and modeling methods, making their simultaneous consideration more challenging. Furthermore, to the best of our knowledge, there is a knowledge gap in regarding the flexibility characteristics of computility center in mitigating fluctuations in PV generation.

In this context, to address the above issues, this paper proposes a computing resources dynamic pooling (CRDP) method to address the fragmentation issue in coordinated scheduling optimization of CC microgrid. This method establishes centrally managed computing resource pool by integrating idle resources through the network, thereby enhancing the flexibility of CC loads. The main contributions of the paper are summarized as follows:

1) A workload-core mapping model is proposed to evaluate the processors occupancy. Specifically, the computility quantification method is proposed to formulate the core operational state matrix, thereby converting workload into power load.
2) A computing resources dynamic pooling method is proposed to integrate remaining available cores based on the core real-time states. Specifically, the computing resource pool is proposed to integrate idle cores continuously. Maximum available cores priority strategy is developed to select the appropriate server type and allocate workloads during CRDP, enhancing CC load responsiveness.
3) A self-regulating CC microgrid coordinated framework is proposed to schedule the CC load to mitigate the volatility of PV generation. Specifically, an adaptive constraint is proposed to set the duration of CRDP and computational workloads according to the PV power. Then a dynamic scheduling method is proposed to leverage energy storage to eliminate the imbalance between power supply and energy demand.

The rest of the paper is organized as follows: Section 2 introduces the problem of coordinated scheduling and fragmented computing resources. Section 3 describes the workload-core mapping model. Section 4 proposes the CRDP method. Section 5 presents a self-regulating CC microgrid coordinated framework. Section 6 verifies the proposed scheduling method. Section 7 makes the conclusions of the paper.

2. Problem description

2.1. Coordinated scheduling for CC microgrid

The high proportion of renewable energy sources, such as distributed PV, connected to the microgrid makes it difficult for traditional flexibility resources to meet the substantial demand caused by the uncertainty of PV generation. As a flexible load with high power consumption and the capability to adjust power demand by modifying workload processing times, the CC has garnered increasing attention. Fig. 1 illustrates a typical microgrid structure, integrating controllable distributed generators, renewable distributed power, energy storage, and local loads. This paper considers the CC as a demand response load within the microgrid.

The microgrid central controller sends scheduling signals to each unit based on PV generation to maintain power balance. When PV generation exceeds the sum of conventional and CC loads, the CC microgrid increases the CC load through demand response signals. It then coordinates the other units to work together, ensuring full consumption of the excess PV power.

However, the CC contains various types of workloads and servers. The CC load generated by workloads during processing is influenced by the dual heterogeneity of the workloads and servers. This situation complicates accurate workload scheduling and hinders the coordination of the control CC load with PV generation. Additionally, shifted workloads are typically processed over extended periods, causing the shifted loads to persist longer. Therefore, the CC must frequently dispatch workloads to respond to PV fluctuations. Meanwhile, the CC microgrids must adjust the output of each unit to maintain power balance.

2.2. Fragmentation of computing resources

In this paper, computing resources refer to the cores of the processors in servers. Generally, the higher the specifications and the more varied the computing resources required by a workload, the greater the likelihood of computing resource fragmentation. In recent years, the rapid development of AI has increased computing resources fragmentation, as the shift from small to large models and from training to deployment has resulted in a growing diversity of AI workloads and higher demand for computility.

Fig. 2(a) illustrates the impact of fragmented computing resources on load scheduling. Fig. 2(a) shows that during regular scheduling, the CC aims to increase the load between 18 h and 20 h. Workload originally scheduled for 6 h to 8 h can be rescheduled for 18 h to 20 h, yet shifting this workload is initially unfeasible due to insufficient cores on Server 1 and Server 2.

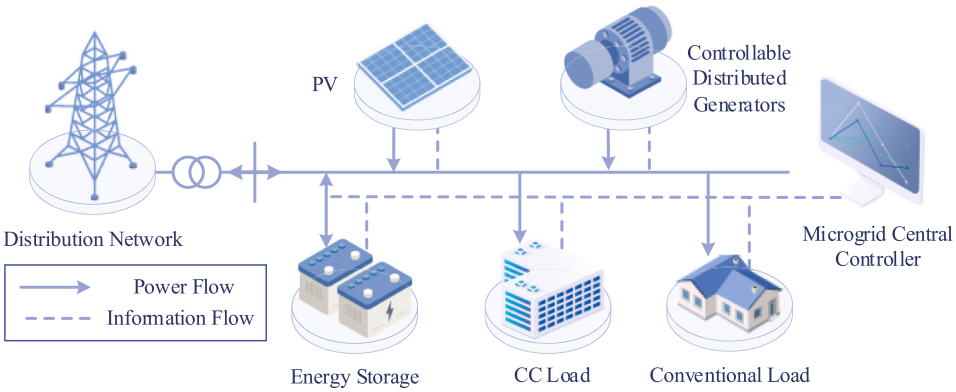


Fig. 1. Structure diagram of microgrid.

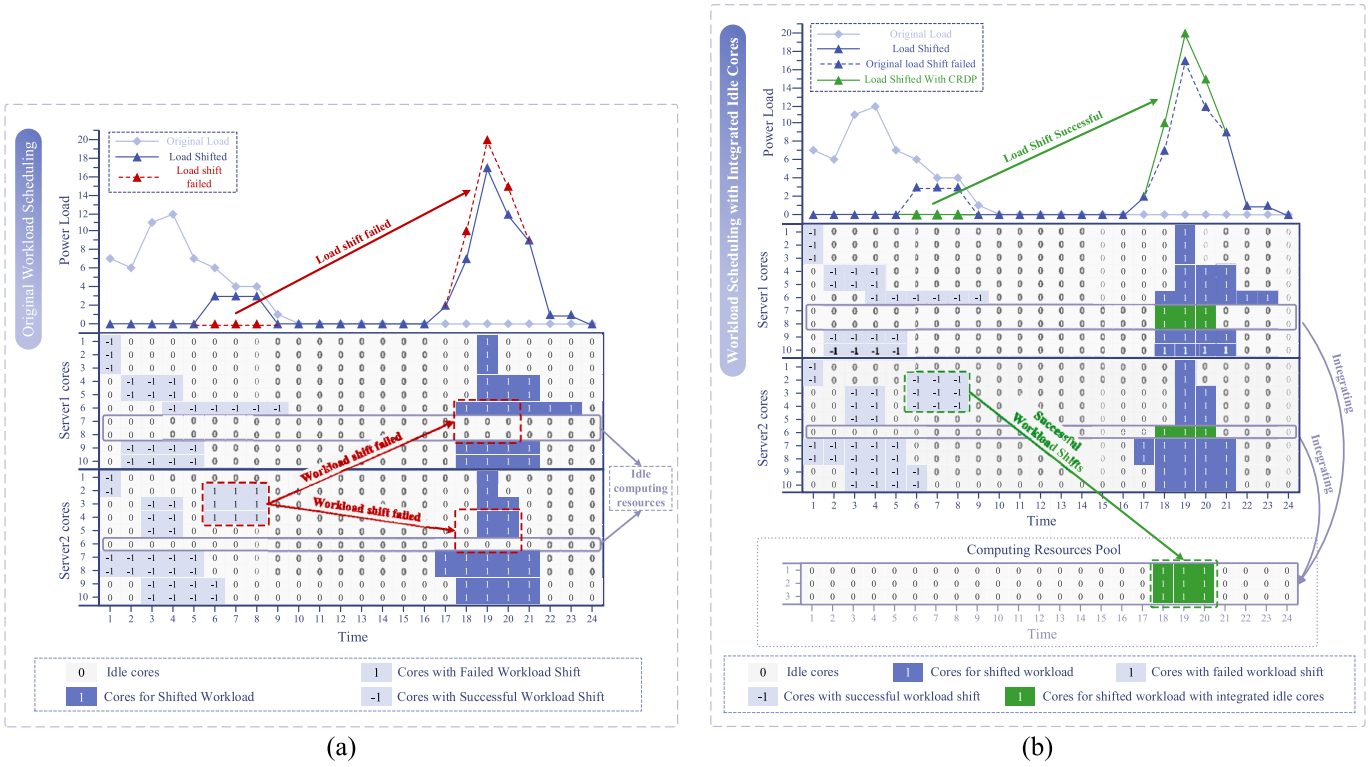


Fig. 2. Comparison of original workload scheduling and workload scheduling with integrated idle cores.

It is possible to consolidate all idle fragmented resources into a computing resource pool. These resources can then be dynamically allocated according to workload demands. This approach overcomes hardware limitations and significantly enhances the load flexibility of the CC. Fig. 2(b) illustrates workload scheduling with integrated idle cores. To demonstrate the effect of CRDP with relative simplicity, Fig. 2 (b) dynamically consolidates only the 7-th and 8-th cores of Server 1 and the 6-th core of Server 2.

With the integration of idle cores, the 7-th and 8-th cores on Server 1 and the 6-th core on Server 2 are integrated into a computing resource pool. This integration frees three cores between 18 h and 20 h, allowing the workload to shift successfully. Consequently, the load of CC rises currently, maximizing the scheduling potential of the CC.

3. Workload-load mapping modeling based on processor core real-time states

The CC load primarily comes from IT equipment, cooling system, network transmission devices, and lighting systems etc. The internal structure of the CC is illustrated in Fig. 3. The IT equipment comprises CPU-based servers and GPU-based servers. The cooling system consists of air compressors and fans. IT equipment accounts for approximately 50 % of the energy usage, cooling systems 30 %, and other equipment, including lighting and network transmission, 20 %.

This section first analyzes workloads and server configurations. Then the processor core operation state matrix is formulated to convert workloads into server utilization. This conversion is based on the processing time of each workload and the number of cores assigned to the workload. And the load of cooling system is modeled according to server heat dissipation and room temperature variations.

3.1. The characterization of workloads

A CC user submits a processing request on the client-side. Then the request generates a workload in the workload scheduling system of CC.

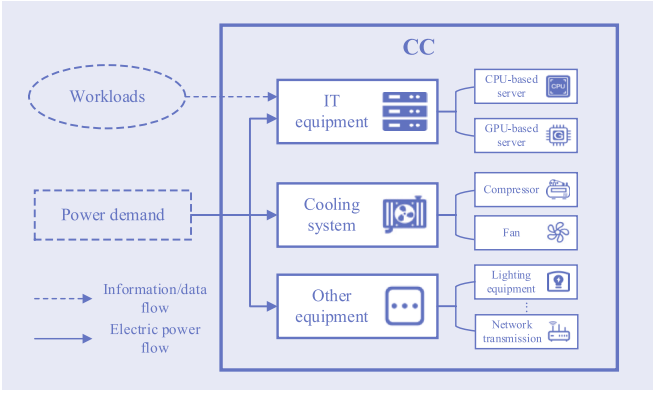


Fig. 3. The architecture of CC.

Workloads are classified into interactive or batch workloads based on the priority and response speed of them.

3.1.1. Interactive workloads

Interactive workloads require that the system or application quickly respond to user inputs or requests. These workloads are also referred to as "latency-sensitive workloads" or "non-transferable workloads." These workloads primarily include web browsing, online trading, cloud gaming, and other real-time user service requests. This type of workload demands priority processing, requiring servers to respond within microseconds to a few seconds. To ensure service quality, this type of workload cannot be scheduled by deferring processing.

3.1.2. Batch workloads

Batch workloads primarily involve executing a set of workloads or operations in a predetermined order without user interaction. These workloads typically run in the background and are not required to respond to user input in real time. They are often referred to as "offline

workloads" or "time-shifted workloads". These workloads primarily include user service requests for AI model training, video color rendering, data processing, and other workloads requiring intensive server processing.

Altering receiving a workload, the workloads scheduling system of CC marks complexity and priority of the batch workload based on its codes logic and sets a processing deadline. Penalties are imposed on CC if the workload exceeds the deadline. Users will also be able to roughly predict processing times based on experience after submitting a batch workload. Therefore, if the batch workload finishes before the deadline, delaying its calculation will not affect user experience or service quality. This deferral time can range from seconds to hours. The feature provides batch workloads processing flexibility with different time scales, allowing the CC to regulate its power demands to be coordinated with the volatility of PV generation through workload dispatching.

3.2. Server load modeling

The workload scheduling system classifies and labels different types of workloads, then assigns them to servers for processing. As the workloads are processed, the server load increases. The variation of server power load is determined by the assigned workload parameters and the server configuration.

3.2.1. CPU-based server

To obtain the processing time of workloads, first calculate the computility of processor in servers. Computility refers to the theoretical maximum number of floating-point operations per second (flops) the server can perform. The formula for the computility of single-core of CPU in server τ is defined as follows:

$$F_{\tau,x}^{\mathrm{C,co}} = N_{\tau}^{\mathrm{C,co}} f_{\tau}^{\mathrm{C,co}} N_{\tau}^{\mathrm{F,co}} FMA_{\tau}^{\mathrm{C,co}} AVX_{\tau}^{\mathrm{C,co}} \Big/ FP_x \tag{1}$$

where $F_{\tau,x}^{\mathrm{C,co}}$ represents the computility of single-core of CPU in server τ. $N_{\tau}^{\mathrm{C,co}}$ denotes the total number of cores of CPU in server τ. $f_{\tau}^{\mathrm{C,co}}$ refers to the single-core frequency of CPU in server τ. $N_{\tau}^{\mathrm{F,co}}$ specifies the number of FMA units of CPU in server τ. $FMA_{\tau}^{\mathrm{C,co}}$ represents the FMA coefficient of CPU in server τ. $AVX_{\tau}^{\mathrm{C,co}}$ refers to the number of vector processing bits in single-core of CPU of server τ during one operation. x indicates the computational precision type, such as FP64, FP32, or INT32 etc. FP_x refers to the number of bits in the workload storage data type x.

This paper assumes that the workload scheduling system evaluates computility, $F_{i,\tau,x}^{\mathrm{B/I}}$, required to process the batch workload or interactive workload i based on its underlying code. The processing time of workload i on CPU of server τ is determined. Thus, the processing time of workload i on CPU of server τ is defined as follows:

$$T_{i,\tau}^{\mathrm{B/I}} = F_{i,\tau,x}^{\mathrm{B/I}} \Big/ N_{i,\tau}^{\mathrm{B/I,co}} F_{\tau,x}^{\mathrm{C,co}} \tag{2}$$

where $T_{i,\tau}^{\mathrm{B/I}}$ represents the processing time of workload i on server τ. $N_{i,\tau}^{\mathrm{B/I,co}}$ represents the number of cores allocated to workload i by server τ.

When the number of workloads is large, the limited computing resources of server prevent all workloads from being processed simultaneously. Some workloads will enter the backup queue and wait for server resources to become available. This paper considers both the waiting time and processing time of the workloads. The response ratio of workload in the backup queue is calculated at each scheduling time. The workload with the highest response ratio is prioritized for processing when resources become available.

$$r_{m}^{\mathrm{B/I,re}} = \left(T_{m,\tau}^{\mathrm{B/I,wa}} + T_{m,\tau}^{\mathrm{B/I}}\right) \Big/ T_{m,\tau}^{\mathrm{B/I}} \tag{3}$$

where $r_{m}^{\mathrm{B/I,re}}$ represents the response ratio of workload m in the backup queue. $T_{m,\tau}^{\mathrm{B/I,wa}}$ represents the waiting time of workload m in the backup queue.

In this paper, the CPU processor core operation state matrix is established to convert the workload into the power load of the CPU-based server.

$$\begin{cases} Z_{\tau} = \left[Z_{\tau,1}^{\mathrm{co}}, Z_{\tau,2}^{\mathrm{co}}, \cdots, Z_{\tau,j}^{\mathrm{co}}, \cdots Z_{\tau,J}^{\mathrm{co}}\right]^{\mathrm{T}} \\ Z_{\tau,j}^{\mathrm{co}} = \left[z_{j1}, z_{j2}, \cdots, z_{jt}, \cdots, z_{jT}\right]_{(1\times T)}, j \in N_{\tau}^{\mathrm{C,co}} \end{cases} \tag{4}$$

where $Z_{\tau,j}^{\mathrm{co}}$ represents the real-time processor core operation state matrix of the j-th core in server τ. z_{jt} is a binary variable, where 0 indicates the j-th core is not processing at time t, and 1 indicates the j-th core is processing at time t. $N_{\tau}^{\mathrm{C,co}}$ represents the number of cores in server τ. $\boldsymbol{Z}_{\tau}$ represents the real-time processor core operation state matrix of server τ. T denotes the total number of times in a day. The operation state of every CPU core of server is accurately recorded through this matrix.

Based on the processor core operation state matrix, the CPU utilization $U_{\tau,t}$ and the power load $P_{\tau,t}^{\mathrm{C}}$ of the CPU in server τ at time t are defined as follows:

$$U_{\tau,t} = \sum_{j=1}^{N_{\tau}^{\mathrm{C,co}}} z_{jt} \Big/ N_{\tau}^{\mathrm{C,co}} \tag{5}$$

$$P_{\tau,t}^{\mathrm{C}} = \left[\frac{1}{2} C_{\tau}^{\mathrm{C}} \left(V_{\tau}^{\mathrm{C}}\right)^2 f_{\tau}^{\mathrm{C,co}} + \alpha_2 \left(T_{\tau,t}^{\mathrm{co,te}}\right)^2\right] U_{\tau,t} + \alpha_1 T_{\tau,t}^{\mathrm{co,te}} + P_{\tau,t}^{\mathrm{C,idl}} \tag{6}$$

where C_{τ}^{C} and V_{τ}^{C} represent the switching capacitance and supply voltage of the CPU in server τ. α_1 and α_2 are the parameters of CPU static power load. $T_{\tau,t}^{\mathrm{co,te}}$ represents the CPU temperature of server τ at time t. $P_{\tau,t}^{\mathrm{C,idl}}$ represents the idle power of CPU, the power consumed when the utilization $U_{\tau,t}$ is zero. At constant voltage, this value remains constant.

The power load of other hardware in the CPU-based server is modeled as follows:

$$P_{\tau,t}^{\mathrm{me}} = \alpha_3 \left(U_{\tau,t}\right)^{\alpha_4} + P_{\tau,t}^{\mathrm{me,st}} \tag{7}$$

$$P_{\tau,t}^{\mathrm{fa}} = P_{\tau}^{\mathrm{fa,ba}} \left(\left(R_{\tau,t}^{\min} + \left(R_{\tau,t}^{\max} - R_{\tau,t}^{\min}\right)\left(U_{\tau,t}\right)^{\alpha_5}\right) \Big/ R_{\tau,t}^{\min}\right)^3 \tag{8}$$

where $P_{\tau,t}^{\mathrm{me}}$ represents the memory power load of server τ at time t. $P_{\tau,t}^{\mathrm{me,st}}$ denotes the static memory power load of server τ at time t. $P_{\tau,t}^{\mathrm{fa}}$ also represents the fan power load of server τ at time t, while $F_{\tau}^{\mathrm{fa,ba}}$ is the base power load of the fan. $R_{\tau,t}^{\max}$ and $F_{\tau,t}^{\min}$ represents the maximum, and minimum rotational speeds of the fan, respectively. α_3 and α_4 are the parameters of memory power load, and α_5 is the parameter of fan power load.

The power load $P_{\tau,t}$ of CPU-based server τ at time t is as follows:

$$P_{\tau,t} = \left(P_{\tau,t}^{\mathrm{C}} + P_{\tau,t}^{\mathrm{me}} + P_{\tau,t}^{\mathrm{fa}} + P_{\tau,t}^{\mathrm{di}}\right) \Big/ \eta^{\mathrm{loss}} \tag{9}$$

where $P_{\tau,t}^{\mathrm{di}}$ is the power load of the hard disk in server τ at time t. η^{loss} represents the energy loss of the server power supply during AC to DC conversion.

3.2.2. GPU-based server

The formula for the computility of single CUDA core in GPU processor k of GPU server υ is as follows:

$$F_{\upsilon,k,x}^{\mathrm{co}} = F_{\upsilon,k,x} \Big/ N_{\upsilon,k,x}^{\mathrm{co}} \tag{10}$$

where $F_{\upsilon,k,x}^{\mathrm{co}}$ is the computility of single CUDA core of type x in the GPU k of server υ. $F_{\upsilon,k,x}$ represents the computility of total CUDA core of type x in the GPU k of server υ. $N_{\upsilon,k,x}^{\mathrm{co}}$ represents the number of CUDA cores of

type x in the GPU k of server υ.

Thus, the processing time of workload i on the GPU server is as follows:

$$T^{\mathrm{B/I}}_{i,\upsilon,k} = F^{\mathrm{B/I}}_{i,\upsilon,x} \Big/ N^{\mathrm{B/I,co}}_{i,x} F^{\mathrm{co}}_{\upsilon,k,x} \tag{11}$$

where $T^{\mathrm{B/I}}_{i,\upsilon,x}$ represents the processing time for workload i on GPU k of server υ.

Similarly, the GPU processor core operation state matrix is established to convert the workload into the power load of the GPU-based server.

$$\begin{cases} Z_\upsilon = \left[Z^{\mathrm{C}}_\upsilon Z^{\mathrm{G}}_\upsilon\right]^{\mathrm{T}}_{N^{\mathrm{co}}_\upsilon \times T} \\ Z^{\mathrm{C}}_\upsilon = \left[Z^{\mathrm{C,co}}_{\upsilon,1}, Z^{\mathrm{C,co}}_{\upsilon,2}, \cdots, Z^{\mathrm{C,co}}_{\upsilon,j}, \cdots, Z^{\mathrm{C,co}}_{\upsilon,J}\right]^{\mathrm{T}}_{J\times T}, j \in N^{\mathrm{C,co}}_\upsilon \\ Z^{\mathrm{C,co}}_{\upsilon,j} = \left[z_{j1}, z_{j2}, \cdots, z_{jt}, \cdots, z_{jT}\right]_{(1\times T)} \\ Z^{\mathrm{G}}_\upsilon = \left[Z^{\mathrm{co}}_{\upsilon,1}, Z^{\mathrm{co}}_{\upsilon,2}, \cdots, Z^{\mathrm{co}}_{\upsilon,k}, \cdots, Z^{\mathrm{co}}_{\upsilon,K}\right]^{\mathrm{T}}_{N^{\mathrm{G,co}}_\upsilon \times T}, k \in N^{\mathrm{pro}}_\upsilon \\ Z^{\mathrm{co}}_{\upsilon,k} = \left[Z^{\mathrm{co}}_{\upsilon,k,1}, Z^{\mathrm{co}}_{\upsilon,k,2}, \cdots, Z^{\mathrm{co}}_{\upsilon,k,x}, \cdots, Z^{\mathrm{co}}_{\upsilon,k,X}\right]^{\mathrm{T}}_{N^{\mathrm{core}}_{\upsilon,k}\times T}, x \in N^{\mathrm{co,ty}}_{\upsilon,k} \\ Z^{\mathrm{co}}_{\upsilon,k,x} = \left[Z^{\mathrm{co}}_{\upsilon,k,x,1}, Z^{\mathrm{co}}_{\upsilon,k,x,2}, \cdots, Z^{\mathrm{co}}_{\upsilon,k,x,l}, \cdots, Z^{\mathrm{co}}_{\upsilon,k,x,L}\right]^{\mathrm{T}}_{L\times T}, l \in N^{\mathrm{co}}_{\upsilon,k,x} \\ Z^{\mathrm{co}}_{\upsilon,k,x,l} = \left[z_{l1}, z_{l2}, \cdots, z_{lt}, \cdots, z_{lT}\right]_{(1\times T)} \end{cases} \tag{12}$$

where $\boldsymbol{Z}_\upsilon$ represents the operation state matrix of total CUDA cores of server υ. N^{co}_υ represents the total number of cores of server υ. Z^{C}_υ and Z^{G}_υ represents the processor core operation state matrix of CPU and GPU in server υ, respectively. $Z^{\mathrm{C,co}}_{\upsilon,j}$ is the processor core operation state matrix of the j-th core in the CPU of server υ. $N^{\mathrm{G,co}}_\upsilon$ represents the number of CUDA cores of GPU in server υ. $N^{\mathrm{pro}}_\upsilon$ represents the number of GPU in server υ. $Z^{\mathrm{co}}_{\upsilon,k}$ represents the operation state matrix of GPU k in server υ. $N^{\mathrm{co}}_{\upsilon,k}$ represents the number of cores of GPU k in server υ. $N^{\mathrm{co,ty}}_{\upsilon,k}$ represents the number of core types of GPU k in the server υ. $Z^{\mathrm{co}}_{\upsilon,k,x}$ represents the operation state matrix of the type x-th core of GPU k in server υ. $N^{\mathrm{co}}_{\upsilon,k,x}$ represents the number of cores of type x of GPU k in server υ. $Z^{\mathrm{co}}_{\upsilon,k,x,l}$ represents the operation state matrix of the tpye x l-th core of GPU k in server υ. z_{lt} is a binary variable, where 0 indicates the l-th CUDA core is not processing at time t, and 1 indicates the l-th CUDA core is processing at time t.

Because the GPU contains various types of CUDA cores, the power load of each type differs. To determine the proportion of power load for each core type relative to the total, the weight of each core type is assigned based on its computility. Based on the CUDA cores operation state matrix, the GPU k utilization $U_{\upsilon,k,t}$ of server υ at time t is formulated as follows:

$$U_{\upsilon,k,t} = \sum_{x=1}^{N^{\mathrm{co,ty}}_{\upsilon,k}} \sum_{l=1}^{N^{\mathrm{co}}_{\upsilon,k,x}} z_{lt} F^{\mathrm{co}}_{\upsilon,k,x} \Big/ N^{\mathrm{co,ty}}_{\upsilon,k} F^{\mathrm{co}}_{\upsilon,k} \tag{13}$$

where $F^{\mathrm{co}}_{\upsilon,k}$ represents the total computility of GPU processor of server υ.

The GPU processors require high-speed global memory to support large-scale parallel processing workloads. Frequent access to large-capacity global memory for reading and writing data results in high GPU memory power load. The GPU power load $P^{\mathrm{G}}_{\upsilon,k,t}$ of GPU server υ is as follows:

$$P^{\mathrm{G}}_{\upsilon,k,t} = P^{\mathrm{co}}_{\upsilon,k,t} + P^{\mathrm{me}}_{\upsilon,k,t} \tag{14a}$$

$$P^{\mathrm{co}}_{\upsilon,k,t} = \beta_{k,1} V^{\mathrm{co}}_{\upsilon,k,t} + \beta_{k,2} V^{\mathrm{co}}_{\upsilon,k,t} f^{\mathrm{co}}_{\upsilon,k,t} \left(\beta_{k,3} + U_{\upsilon,k,t}\right) \tag{14b}$$

$$P^{\mathrm{me}}_{\upsilon,k,t} = \beta_{k,4} V^{\mathrm{me}}_{\upsilon,k,t} + \beta_{k,5} V^{\mathrm{me}}_{\upsilon,k,t} f^{\mathrm{me}}_{\upsilon,k,t} \left(\beta_{k,6} + U_{\upsilon,k,t}\right) \tag{14c}$$

where $P^{\mathrm{co}}_{\upsilon,k,t}$ represents the power load of the CUDA cores of GPU k in server υ at time t. $P^{\mathrm{me}}_{\upsilon,k,t}$ is the power load of the global memory of GPU k in server υ at time t. $V^{\mathrm{co}}_{\upsilon,k,t}$ and $V^{\mathrm{me}}_{\upsilon,k,t}$ denote the voltages of the CUDA core and global memory of GPU k in server υ at time t. $f^{\mathrm{co}}_{\upsilon,k,t}$ and $f^{\mathrm{me}}_{\upsilon,k,t}$ indicate the frequencies of the CUDA core and global memory of GPU k in server υ at time t. $\beta_{k,1}$ through $\beta_{k,6}$ are parameters related to the power load of GPU k.

The power load of other hardware devices in GPU-based server υ is essentially the same as that of the CPU-based server and follows the same as shown in Eqs. (5)–(8).

Overall, the power load $\mathrm{P}_{\upsilon,t}$ of GPU-based server υ at time t is as follows:

$$P_{\upsilon,t} = \left(\sum_{k=1}^{N^{\mathrm{G}}_\upsilon} P^{\mathrm{G}}_{\upsilon,k,t} + P^{\mathrm{C}}_{\upsilon,t} + P^{\mathrm{me}}_{\upsilon,t} + P^{\mathrm{fa}}_{\upsilon,t} + P^{\mathrm{di}}_{\upsilon,t}\right) \Big/ \eta^{\mathrm{loss}} \tag{15}$$

3.3. Cooling system load modeling

Servers, lighting, and other equipment in the CC generate substantial heat, increasing the temperature of the server room. The cooling system is used to regulate the temperature of server room in CC. This system accounts for approximately 30 % of the total power load of the center.

The thermal dynamic equilibrium process occurs continuously in the server room. Heat is generated continuously by the server and other equipment. Additionally, a portion of the heat is transferred into the room through the walls due to the temperature difference between indoors and outdoors. Meanwhile, heat also dissipated. For example, an air conditioning unit in a server room generates cooling to offset some heat. A portion of the heat is also consumed as it is converted into temperature. The remaining unconsumed heat is carried over to the next moment. Therefore, the heat conservation equation in the server room of the CC is as follows:

$$Q^{\mathrm{en}}_t = Q^{\mathrm{be}}_t + Q^{\mathrm{se}}_t + Q^{\mathrm{ou}}_t + Q^{\mathrm{ot}}_t - Q^{\mathrm{te}}_t - Q^{\mathrm{coo}}_t \tag{16a}$$

$$Q^{\mathrm{be}}_t = Q^{\mathrm{en}}_{t-1} \tag{16b}$$

where Q^{en}_t represents the heat in the server room at the end of time t. Q^{be}_t represents the heat in the server room at the begin of time t. Q^{se}_t is the heat generated by the server during time t. Q^{ou}_t represents the amount of heat absorbed by the indoor air from the outside through the walls during time t due to the heat transfer. Q^{ot}_t is the heat generated by equipment other than the servers during time t, such as lighting. This value is generally constant. Q^{te}_t represents the heat converted to temperature during time t. Q^{coo}_t is the heat offset by the cooling system during time t.

The heat generated by the server device at time t is as follows:

$$Q^{\mathrm{se}}_t = \eta^{\mathrm{se}} \left(\sum_{\tau=1}^{N_\tau} P_{\tau,t} + \sum_{\upsilon=1}^{N_\upsilon} P_{\upsilon,t}\right) \tag{16c}$$

where η^{se} represents the heat conversion efficiency of the server equipment.

The heat Q^{te}_t converted to temperature and the resulting temperature change ΔT^{te}_t at time t are as follows:

$$\Delta T^{\mathrm{te}}_t = \eta^{\mathrm{te}} \left(Q^{\mathrm{be}}_t + Q^{\mathrm{se}}_t + Q^{\mathrm{ou}}_t + Q^{\mathrm{ot}}_t\right) / m^{\mathrm{air}} c^{\mathrm{air}} \tag{16d}$$

$$T^{\mathrm{te,in,en}}_t = T^{\mathrm{te,in,be}}_t + \Delta T^{\mathrm{te}}_t \tag{16e}$$

$$T^{\mathrm{te,in,en}}_t = T^{\mathrm{te,in,be}}_{t-1} \tag{16f}$$

where η^{te} represents the ratio of heat converted to temperature. m^{air} is the mass of air in the server room. c^{air} is the specific heat capacity of air. $T^{\mathrm{te,in,be}}_t$ and $T^{\mathrm{te,in,en}}_t$ are the indoor temperature of the server room at the beginning and end of time t, respectively.

The cooling load primarily consists of two components: the air conditioning compressor load and the fan load. The compressor load is determined by the amount of heat it offsets and cooling efficiency of the air conditioning compressor.

$$P_t^{\text{comp}} = Q_t^{\text{coo}} / \left(\eta^{\text{COP}} \Delta t\right) \tag{16g}$$

$$\eta^{\text{COP}} = 0.0068 T^{\text{set2}} + 0.0008 T^{\text{set}} + 0.458 \tag{16h}$$

where P_t^{comp} represents the load of the air conditioner compressor, and η^{COP} is its cooling efficiency. A higher η^{COP} indicates greater cooling efficiency of the refrigeration unit. This value of air conditioning compressor is not fixed and fluctuates with changes in the temperature environment. In this paper, we use the η^{COP} model, quantified based on the set temperature T^{set}.

The load of fan is proportional to the cube of the fan blade speed:

$$P_t^{\text{coo,fa}} = P_t^{\text{coo,fa,ba}} \left(R_t^{\text{fa}} / R_t^{\min}\right)^3 \tag{16i}$$

where $P_t^{\text{coo,fa}}$ represents the load of fan, $P_t^{\text{coo,fa,ba}}$ is the selected fan load reference, $R_t^{\min}$ and R_t^{fa} denote the minimum fan speed and real-time fan speed, respectively.

Thus, the power load for cooling $\mathrm{P}_t^{\text{coo}}$ the CC is:

$$P_t^{\text{coo}} = P_t^{\text{comp}} + P_t^{\text{coo,fa}} \tag{16j}$$

$$P^{\text{coo,min}} \le P_t^{\text{coo}} \le P^{\text{coo,max}} \tag{16k}$$

where $P^{\text{coo,min}}$ and $P^{\text{coo,max}}$ represent the minimum and maximum load of the cooling system, respectively.

3.4. CC load modeling

In addition to the server equipment load and cooling system power load, part of the CC load consists of lighting equipment, network transmission equipment, and other components. Thus, the CC load at time t is as follows:

$$P_t^{\text{CC}} = \sum_{\tau=1}^{N^{\text{C}}} P_{\tau,t} + \sum_{\upsilon=1}^{N^{\text{G}}} P_{\upsilon,t} + P_t^{\text{coo}} + P_t^{\text{ill}} + P_t^{\text{ot}} \tag{17a}$$

where P_t^{ill} represents the lighting power load. P_t^{ot} represents the power load of other devices. N^{C} and N^{G} represent the number of CPU servers and GPU servers in the CC, respectively.

PUE is the metric for evaluating the energy efficiency of CC. The energy efficiency level η^{PUE} of existing CC should be below 1.3. The energy efficiency level of CC is constrained as follows:

$$\eta^{\text{PUE}} = \left(P_t^{\text{coo}} + \sum_{\tau=1}^{N^{\text{C}}} P_{\tau,t} + \sum_{\upsilon=1}^{N^{\text{G}}} P_{\upsilon,t}\right) \Bigg/ \left(\sum_{\tau=1}^{N^{\text{C}}} P_{\tau,t} + \sum_{\upsilon=1}^{N^{\text{G}}} P_{\upsilon,t}\right) \tag{17b}$$

4. Computing resources dynamic pooling method

To address the impact of fragmented computing resources, we propose a CRDP method based on maximum available cores priority. This method integrates heterogeneous resources such as CPU and GPU chips using cloud computing, creating centrally managed resource pools. Then these resources are scheduled and allocated according to workload demand. In this paper, we assume that the internal network communication of the CC meets the requirements for resource pooling. As a result, the resource pool can allocate the necessary computational resources based on workload demands, maximizing resource utilization and enhancing load scheduling flexibility.

To accurately represent the allocation of the computing resources pool, this paper proposes the CRDP operation state matrix, $\boldsymbol{Z}^{\text{po}}$:

$$\begin{cases} Z^{\text{po}} = \left[Z_1^{\text{po}}, Z_2^{\text{po}}, \cdots, Z_t^{\text{po}}, \cdots, Z_T^{\text{po}}\right]_{N^{\text{co}} \times T} \\ Z_t^{\text{po}} = \left[Z_t^{\text{idl}} \;\; Z_t^{\text{occ}}\right]^{\text{T}} \\ Z_t^{\text{idl}} = [z_{1t}, z_{2t}, \cdots, z_{et}, \cdots, z_{Et}], e \in N_t^{\text{co,idl}} \\ Z_t^{\text{occ}} = [z_{1t}, z_{2t}, \cdots, z_{ut}, \cdots, z_{Ut}], u \in N_t^{\text{co,occ}} \end{cases} \tag{18}$$

where Z_t^{idl} and Z_t^{occ} represent the core idle state matrix and core occupied state matrix, respectively at time t. z_{et} is a binary variable, where 0 indicates that the e-th core at time t is idle, and 1 indicates it is occupied. To differentiate from z_{et} and facilitate server power consumption calculations, the value of z_{ut} in Z_t^{occ} is set to 2, indicating that the u-th core is occupied at time t. $N_t^{\text{co,idl}}$ and $N_t^{\text{co,occ}}$ represent the total number of cores in the idle and occupied states, respectively, at time t. N^{co} represent the total number of cores in CC. The CRDP operation state matrix provides a clear representation of resource allocation at each time.

After assigning workloads at time t, for each server, record the number of idle cores $N_t^{\text{co,idl}}$ and occupied cores $N_t^{\text{co,occ}}$ based on the core operation state matrix $\boldsymbol{Z}_\tau$ or $\boldsymbol{Z}_\upsilon$ for that server. Subsequently, idle cores of same-type servers are separately integrated into a centrally managed pool of computing resources, and the operation state matrix $\boldsymbol{Z}^{\text{po}}$ for the CRDP is updated according to the number of cores in different states. CC assigns the corresponding cores in the computing resource pool to workloads based on the demand of the workloads and the number of idle cores remaining in each type of server. CC updates $\boldsymbol{Z}^{\text{po}}$ to reflect whether each core is occupied or not. At this point, CC's idle computing resources can be fully utilized and its regulation potential is thoroughly explored. After the computing resource pool is allocated, changes in z_{jt} in the computing resource pool matrix update the corresponding server operation state matrix $\boldsymbol{Z}_\tau$ or $\boldsymbol{Z}_\upsilon$. The workload allocated to the resource pool is converted into the load of the CC based on the CC model from Section 2.

Algorithm: CRDP based on maximum available cores priority strategy

1: Record the number of available cores at time t after normal allocation to formulate the $\boldsymbol{Z}^{\text{po}}$.
2: **if** $N_t^{\text{B}} \neq 0$ or $N_t^{\text{B,bac}} \neq 0$ **then**
3: Place workloads scheduled for processing at that moment into the backup queue.
4: Sort the workloads in the backup queue in descending order of response ratio. (i.e., Workloads with higher response ratios are allocated first.).
5: **for** each workload m in backup queue **do**
6: Identify the server type with the maximum available cores according to $\boldsymbol{Z}^{\text{po}}$.
7: **if** the server type with the most remaining cores differs from the one originally assigned to workload m **then**
8: Update workload processing time $T_{i,\tau}^{\text{B/I}}$ or $T_{i,\upsilon,k}^{\text{B/I}}$ according to Eqs. (2) and (11).
9: Allocate workload m to the server τ or ν which has the most available cores.
10: Update the $\boldsymbol{Z}^{\text{po}}$、$\boldsymbol{Z}_\tau$ and $\boldsymbol{Z}_\upsilon$.
11: **End**

5. Self-regulating CC microgrid coordinated framework

5.1. Microgrid modeling

5.1.1. Controllable distributed generators

In microgrid, controllable distributed generators mainly include microturbine and fuel cells, etc. Their power generation costs can be represented by a linear function. This paper considers the microgrid with microturbine, and its cost function is given as follows:

$$C_t^{\mathrm{MI}} = \left[aP_t^{\mathrm{MI}} + b\right]\Delta t \tag{19a}$$

$$P^{\mathrm{MI,min}} \le P_t^{\mathrm{MI}} \le P^{\mathrm{MI,max}} \tag{19b}$$

where C_t^{MI} is the power generation cost of the microturbine during time t. The coefficients a and b represent cost parameters. P_t^{MI} is the output power of the microturbine during time t. The dispatch step length Δt is 1 h. $P^{\mathrm{MI,max}}$ and $P^{\mathrm{MI,min}}$ represent the maximum and minimum output power of the microturbine. These values are limited by its rated power and minimum load rate.

5.1.2. Energy storage

The operational cost of energy storage includes the cost of initial investment and the cost of operational and maintenance. The average charging and discharging cost $CS\ t$ for time t during the investment payback period is represented as:

$$C_t^{\mathrm{S}} = K^{\mathrm{S}}\left[P_t^{\mathrm{S,ch}}\eta^{\mathrm{S}} + P_t^{\mathrm{S,dis}}/\eta^{\mathrm{S}}\right]\Delta t \tag{20a}$$

where K^{S} represents the discounted unit charging and discharging cost. $P_t^{\mathrm{s,sh}}$ and $P\ \mathrm{S,dis}\ t$ represent the charging and discharging power of the energy storage at time t, respectively. η^{S} is the charging and discharging efficiency of energy storage. The energy storage must meet the constraints during operation:

$$0 \le P_t^{\mathrm{S,dis}} \le U_t^{\mathrm{S}} P^{\mathrm{S,max}} \tag{20b}$$

$$0 \le P_t^{\mathrm{S,ch}} \le \left(1 - U_t^{\mathrm{S}}\right) P^{\mathrm{S,max}} \tag{20c}$$

$$\eta^{\mathrm{S}} \sum_{t=1}^{N^{\mathrm{T,sch}}} \left(P_t^{\mathrm{S,ch}}\Delta t\right) - \left(\sum_{t=1}^{N^{\mathrm{T,sch}}} \left(P_t^{\mathrm{S,dis}}\Delta t\right)\right) \Big/ \eta^{\mathrm{S}} = 0 \tag{20d}$$

$$E^{\mathrm{S,min}} \le E^{\mathrm{S}}(0) + \eta^{\mathrm{S}} \sum_{t'=1}^{t} \left(P_{t'}^{\mathrm{S,ch}}\Delta t\right) - \left(\sum_{t'=1}^{t} \left(P_{t'}^{\mathrm{S,dis}}\Delta t\right)\right) \Big/ \eta^{\mathrm{S}} \le E^{\mathrm{S,max}} \tag{20e}$$

Eqs. (20b) and (20c) define the power constraints for charging and discharging energy storage. $P^{\mathrm{S,max}}$ is the maximum allowable charging and discharging power. U_t^{S} represents the charging and discharging state, where 1 indicates discharging and 0 indicates charging. Eq. (20d) ensures that the energy storage capacity remains equal at the start and end of the scheduling period, which facilitates cyclic scheduling. $N^{\mathrm{T,sch}}$ refers to the scheduling period, set to 24 h. Eq. (20e) defines the remaining capacity constraints for each time, where $E_{\mathrm{S}}(0)$ is the initial capacity, and $E^{\mathrm{S,max}}$ and $E^{\mathrm{S,min}}$ are the maximum and minimum allowable capacities during the scheduling process. This constraint aims to prevent overcharging and over-discharging to extend the service life of the energy storage system.

5.1.3. Demand response load

In this paper, CC load is dispatched as demand response load. When the PV generation is high, the microgrid needs to fully consume the PV by load dispatch through the demand response incentive CC. Therefore, the dispatch cost in time t is:

$$C_t^{\mathrm{DR}} = \chi^{\mathrm{DR}} P_t^{\mathrm{DR}} \tag{21}$$

where C_t^{DR} represents the microgrid dispatch cost for CC at time t. χ^{DR} is the unit dispatch cost of CC load. P_t^{DR} is the CC load dispatch power at time t.

5.1.4. Distribution network exchange power

When the power generation units in the microgrid do not meet the load demand, electricity must be purchased from the distribution network. Conversely, the microgrid can sell surplus electricity to the distribution network to generate revenue. The exchange of power between the microgrid and the distribution network must meet the following balance constraint:

$$P_t^{\mathrm{M,pur}} - P_t^{\mathrm{M,sel}} = P_t^{\mathrm{S,ch}} + P_t^{\mathrm{CC}} + P_t^{\mathrm{DR}} + P_t^{\mathrm{L}} + P_t^{\mathrm{PV,cur}} - P_t^{\mathrm{MI}} - P_t^{\mathrm{S,dis}} - P_t^{\mathrm{PV}} \tag{22a}$$

$$0 \le P_t^{\mathrm{M,pur}} \le U_t^{\mathrm{M}} P^{\mathrm{M,max}} \tag{22b}$$

$$0 \le P_t^{\mathrm{M,sell}} \le \left(1 - U_t^{\mathrm{M}}\right) P^{\mathrm{M,max}} \tag{22c}$$

where $P_t^{\mathrm{M,pur}}$ and $P_t^{\mathrm{M,sel}}$ represent the power purchased from and sold to the distribution network by the microgrid during time t, respectively. This paper considers only the case where the renewable distributed power sources are PV. $P_t^{\mathrm{PV,cur}}$ denote the PV curtailment at time t. P_t^{L} and P_t^{PV} represent the conventional load power and PV generation within the microgrid during time t. $P^{\mathrm{M,max}}$ represents the maximum exchange power between the microgrid and the distribution network. U_t^{M} indicates the purchasing or selling state of the microgrid, where a value of 1 means purchasing electricity from the grid, and a value of 0 means selling electricity to the grid. The exchange cost C_t^{M} during time t is represented as follows:

$$C_t^{\mathrm{M}} = \left(\lambda_t^{\mathrm{pur}} P_t^{\mathrm{M,pur}} - \lambda_t^{\mathrm{sel}} P_t^{\mathrm{M,sel}}\right)\Delta t \tag{22d}$$

where λ_t^{pur} and λ_t^{sel} represents the day-ahead purchase and sale tariffs in the distribution grid.

5.1.5. PV curtailment

When microgrid cannot fully consume the power generated by PV, the excess energy is abandoned or wasted. To enhance the capacity of power utilization in microgrid, this paper assigns a penalty cost $C_t^{\mathrm{PV,cur}}$ to unutilized of PV generation:

$$C_t^{\mathrm{PV,cur}} = \sigma^{\mathrm{PV}} P_t^{\mathrm{PV,cur}} \tag{23}$$

where σ^{PV} represents the cost per unit of abandoned light penalties.

5.2. Coordinated scheduling framework

In the collaborative scheduling process, the microgrid first formulates an hourly power purchase/sale plan with the higher-level grid based on PV generation and residential load profiles. Subsequently, it compensates for source-load imbalances by adjusting microturbine outputs at 15-min intervals. Finally, per-minute refinements are achieved through coordinated energy storage charging/discharging cycles and dynamic workload adjustments on the CC side to modulate its load demand. Due to differing time scales among various loads, specific parameters must be defined to aid scheduling prior to its initiation. The parameters t^{1440}, t^{95}, and t^{24} represent distinct time scales within a single day. The identifier ID^{CRDP} determines the enablement of CRDP, where $ID^{\mathrm{CRDP}} = 1$ indicates CRDP is enabled, and $ID^{\mathrm{CRDP}} = 0$ indicates it is disabled. This binary parameter is defined by the relationship between PV generation and residential load demand, assuming a value of 1 when PV generation exceeds the load and 0 otherwise.

At the start of each t1440 time period, CC evaluates whether the number of backup queues $N_{t1440}^{\mathrm{B/I,bac}}$ exceeds zero. If $N_{t1440}^{\mathrm{B/I,bac}} > 0$, workloads in the backup queue are prioritized for processing. Otherwise, current workloads are processed directly. If insufficient computing resources are

available to handle the workload during scheduling, CC defers the workload by placing it in the backup queue. Once the workload is distributed, immediately calculate the server load.

When excess PV generation occurs, the microgrid uses incentive-based demand response to increase the CC load by adjusting workload processing times, facilitating PV consumption. To enhance scheduling flexibility, CRDP is employed by CC to aggregate idle cores into the centrally managed computing resource pool. CC then checks for queued workloads in the backup queue. If such workloads exist, cores from the computing resource pool are allocated to facilitate their processing. During core allocation, CC evaluates if the workload's processing end time exceeds the CRDP end time. If so, additional cores are allocated to ensure the workload's processing time aligns with the CRDP period. Once core allocation for queued workloads is complete, server utilization increases compared to scenarios without CRDP, thereby raising the total CC load. If no workloads are present in the backup queue, arriving workloads are allocated in the standard manner. Under incentivized demand response, the CC processes more workloads at lower tariffs. The microgrid uses the additional load for PV consumption, creating a mutually beneficial outcome. After the scheduling of CC is completed, if PV generation remains unconsumed, the microgrid reduces PV curtailment by using energy storage.

When PV generation is low, the microgrid can obtain power through energy storage discharge, microturbine, or power purchases from the distribution network to support regular loads and the CC. By scheduling workloads in the backup queue to calculate during periods of high PV generation, the CC shifts the load that would have occurred during low PV generation to times of higher generation. This reduces the CC load during periods of low PV generation, relieving pressure on the microgrid. This strategy enables coordinated scheduling of CC microgrid by CRDP. Fig. 4 illustrates the specific process.

5.2.1. Objective function

To verify the effectiveness of the proposed strategy, this paper uses the minimization of the daily operating cost of microgrid as the objective function:

$$\min \sum_{t=1}^{N^{\mathrm{T}}} \left[C_t^{\mathrm{MI}} + C_t^{\mathrm{S}} + C_t^{\mathrm{DR}} + C_t^{\mathrm{M}} + C_t^{\mathrm{PV,cur}} \right] \tag{24}$$

5.2.2. Restrictive condition

The internal constraints of the CC include workload, server hardware, and cooling power constraints, etc.

Interactive loads require timely processing. The number of interactive workloads processed by CC during time t must meet the required conditions:

$$N_t^{\mathrm{I,arr}} = N_t^{\mathrm{I,com}} \tag{25}$$

where $N_t^{\mathrm{I,arr}}$ represents the number of interactive workloads arriving at the CC at time t. $N_t^{\mathrm{I,com}}$ represents the number of interactive workloads completed by the CC at time t.

Batch workloads can delay processing. To ensure quality of service, they must finish before the deadline. Batch workloads have the following constraints:

$$T_i^{\mathrm{B,com,min}} \le T_i^{\mathrm{B,com}} \le T_i^{\mathrm{B,com,max}} \tag{26a}$$

$$T_i^{\mathrm{B,com,min}} = T_i^{\mathrm{B,arr}} + T_i^{\mathrm{B}} \tag{26b}$$

where $T_i^{\mathrm{B,com,min}}$ represents the minimum processing time of the batch workload. $T_i^{\mathrm{B,com,max}}$ represents the maximum processing time of the batch workload, also known as the deadline time. $T_i^{\mathrm{B,arr}}$ represents the time when the batch workload arrives at the CC.

The microgrid must satisfy the constraints in Eq. (19b), Eqs. (20b) -(20e), and Eqs. (22b) and (22c).

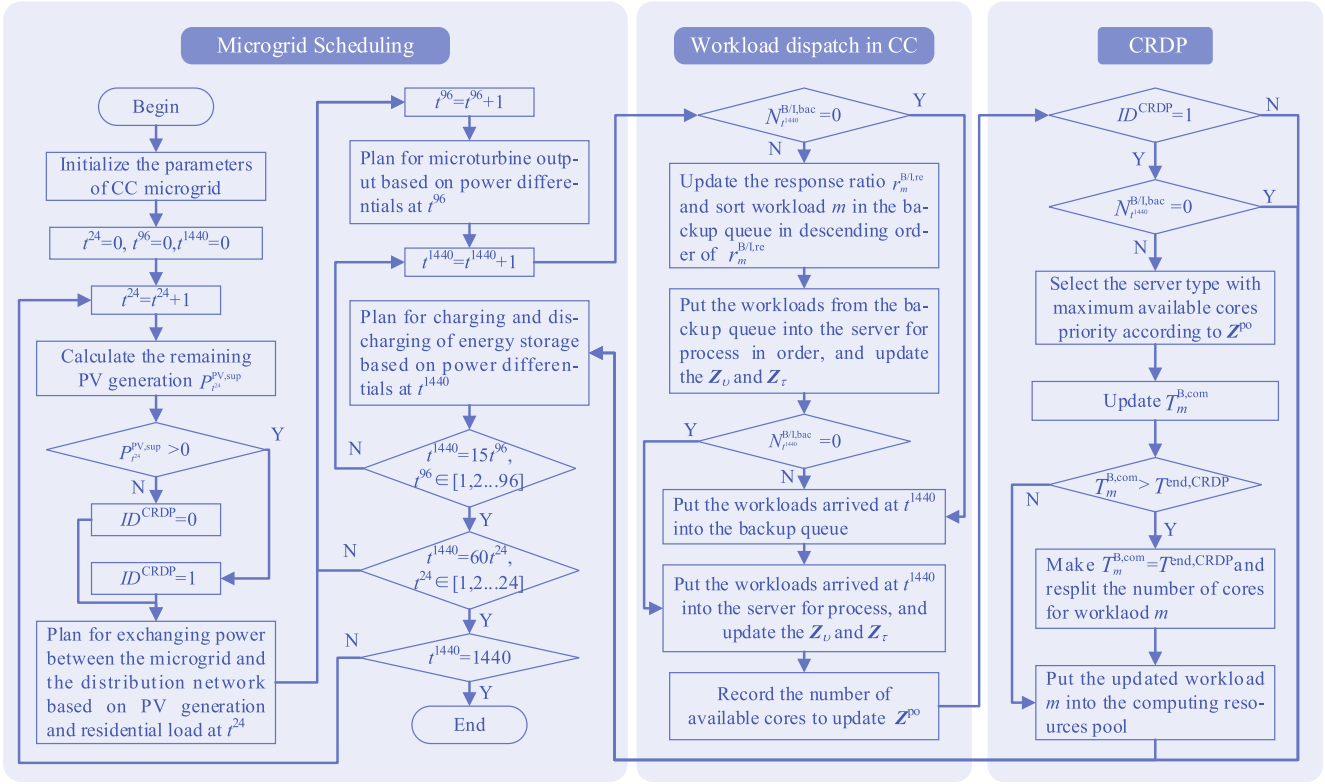


Fig. 4. Flow chart of self-regulating CC microgrid coordinated framework.

6. Case study

In this section, the microgrid in Fig. 1 serves as the research example to verify the effectiveness of proposed scheduling method. The proposed model is implemented in the YALMIP optimization toolbox with MATLAB R2022b and solved by IBM ILOG CPLEX 12.10. The numerical experiments were carried out on a computer with an Intel Core i5-9300H CPU @ 2.40GHz.

6.1. Experimental settings

6.1.1. Parameters of CC

Server and workload data for the CC are sourced from the real production workload trace from Alibaba. The detailed parameters of servers and workload are given in the Appendix.

The initial indoor temperature is set to 23.5 °C, with an upper limit of 30 °C and a lower limit of 18 °C. The specific heat capacity of air c^{air} is 1005 J/(kg·°C). The heat conversion efficiency of the server equipment η^{se} is 0.99. Outdoor temperature changes are given in the Appendix.

6.1.2. Parameters of microgrid

The operating parameters of each microgrid unit, the conventional load, and the PV generation are given in the Appendix. The day-ahead transaction tariff for power exchange between the distribution grid and the microgrid is based on the regional tariff in East China. The detailed tariff is given in the Appendix. The selling tariff of the microgrid is lower than the purchasing tariff, due to this microgrid is PV-based. It is affected by weather and other factors, resulting in a less stable power supply compared to the distribution grid.

6.1.3. Case settings

To verify the economics of the CC microgrid scheduling strategy considering CRDP, two sets of cases are set up for comparative analysis.

Case A1. CC microgrid without CRDP and energy storage.

Case A2. CC microgrid with CRDP.

Case B1. CC microgrid with energy storage.

Case B2. CC microgrid with CRDP and energy storage.

The operational parameters of the microgrid and CC in each case use the parameters set above.

6.2. Result analysis

6.2.1. Analysis of optimal CC microgrid scheduling results

Table 1 shows the total cost of CC microgrid operation and the PV curtailment in the four cases. Table 1 indicates that both PV curtailment and total cost decrease after considering CRDP.

To analyze the effect of CRDP on microgrid operation costs in detail, two scheduling cases, Case B1 and Case B2, are selected for comparative analysis. The scheduling optimization results are shown in Figs. 5–7. Fig. 5(a) and Fig. 5(b) show the average utilization rate of all CC servers in two cases, respectively. “M32” denotes a server equipped with V100M32 processor. Fig. 5(c) compares the backup queue of the CC in both cases. Fig. 5(d) shows the PV generation and comparison of scheduling results of CC load. Figs. 6(a) and 6(b) display the output power of the microturbine, and the power purchased or sold by the microgrid to the distribution grid. A positive value indicates power purchase, while a negative value indicates power sale. Figs. 7(a) and 7(b) present the charging and discharging power, along with the SOC states of the energy storage. Positive values represent charging, while negative values represent discharging.

Table 1
Comparison of PV curtailment and operating costs.

	PV curtailment (kW)	Operating costs ($)	Relative cost reduction rate
Case A1	277.66	4,975,600.09	–
Case A2	194.61	4,620,783.08	7.13 %
Case B1	65.96	4,666,032.58	–
Case B2	0	4,320,914.78	7.39 %

Case B1. CC microgrid with energy storage.

Fig. 5(d) shows that between 1 h and 5 h, PV generation is zero, and the microgrid load relies entirely on the microturbine and distribution grid supply. During this period, the microturbine operates at minimum output, as shown in Fig. 6(a), because the distribution grid purchase tariff is lower than the microturbine generation cost. Due to the rise in the distribution network tariff after 8 h, the energy storage, as shown in Fig. 7(a), is charged between 1 h and 5 h to store power during off-peak hours, preparing for shortages during peak hours.

At 6 h, PV generation starts increasing, and the microgrid load shifts from relying on microturbine and the distribution grid to a combination of PV, microturbine, and energy storage. Between 6 h and 7 h, the load is supplied by the distribution grid, PV, and microturbine due to low PV generation and unchanged grid prices. During this time, energy storage continues charging. After 8 h, the purchasing tariff increases from 0.065 $/kWh to 0.125$/kWh, exceeding the unit cost of microturbine generation and energy storage discharge. As a result, the microgrid stops purchasing electricity from the distribution grid between 8 h and 9 h, as shown in Fig. 6(b). The microgrid load is powered by PV, microturbine, and energy storage. After 10 h, PV generation increases significantly, and Case B1 requires additional load by dispatching CC to consume excess PV. Fig. 5(a) shows many servers operating at over 70 % utilization during this time, while Fig. 5(c) indicates many workloads are waiting in the backup queue. This suggests that fragmented computing resources in servers of CC are underutilized, and additional loads cannot be shifted from later periods to consume PV. After 10 h, PV generation increases, and the microgrid consumes PV by charging energy storage. In Fig. 7(a), energy storage charges at 10 h, 12 h, and 13 h to store excess PV generation. Meanwhile, the microgrid sells electricity to the distribution grid between 10 h and 15 h to utilize PV and reduce operating costs. From 16 h to 18 h, due to decreased PV generation and high electricity prices, the output of the microturbine in Fig. 6(a) increases, supplying the microgrid load along with PV.

Between 19 h and 21 h, PV generation is nearly zero. As the distribution grid tariff rises from 0.059$/kWh to 0.091$/kWh, the microgrid sells excess electricity from the microturbine and energy storage to the grid. From 22 h to 23 h, the grid tariff increases to 0.188$/kWh. Therefore, the microgrid utilizes energy storage and the microturbine, which have lower generation costs, for supply. At 24 h, the grid tariff decreases, and most loads are supplied by the distribution grid. Due to limited transmission capacity, a small portion of the load is still supplied by the microturbine.

Case B2. CC microgrid with CRDP and energy storage.

The only difference between Case B2 and Case B1 is whether CC adopts CRDP to improve scheduling flexibility. Therefore, differences in microgrid scheduling between the two cases are not evident until 10 h. As shown in Fig. 6, both Case B1 and Case B2 are primarily supplied by the distribution grid and microturbine from 1 h to 7 h, and by PV, microturbine, and energy storage at 8 h. Energy storage charges between 1 h and 7 h and discharges at 8 h.

After 9 h, the microgrid shifts loads through demand response by scheduling the CC due to excess PV generation. Fig. 5(c) shows that, compared to Case B1, the number of workloads in backup queue of Case B2 is zero between 9 h and 15 h. In Fig. 5(b), the utilization rate of GPU-based servers P100 and M32 in Case B2 increases from 70 % to 100 %, indicating that CC uses CRDP to allocate all backup workloads into the

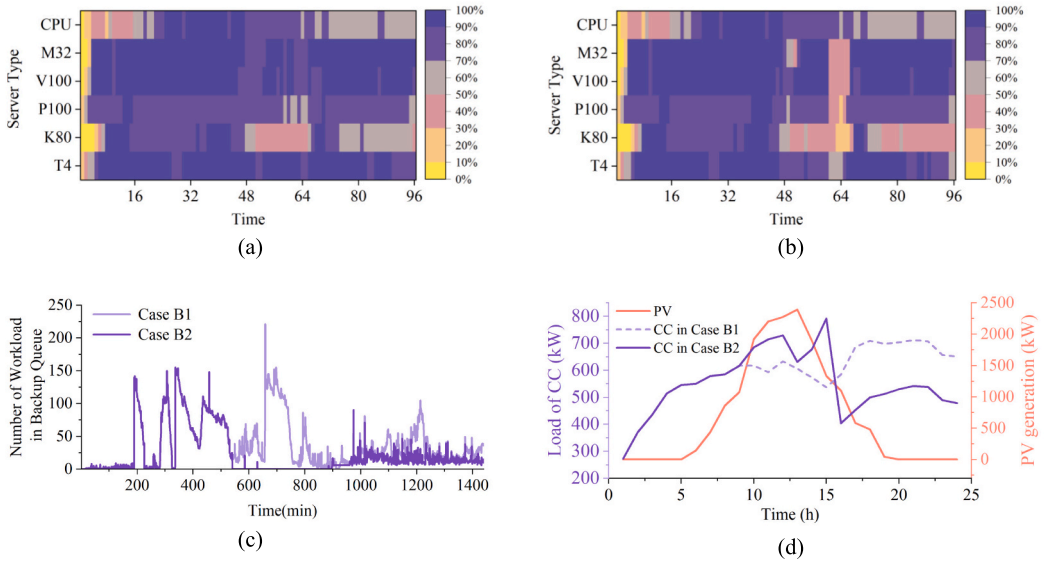


Fig. 5. Comparison of scheduling results of CC.

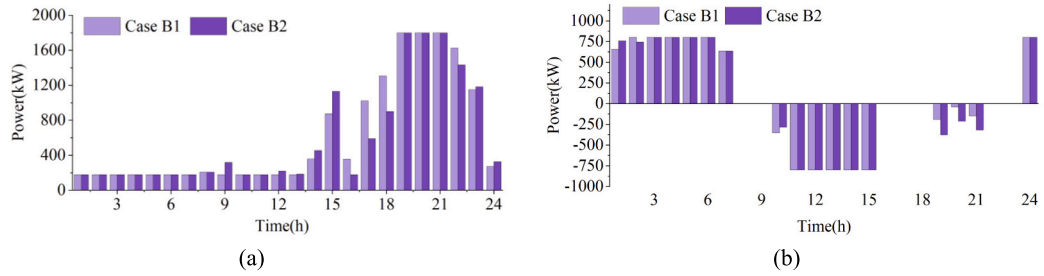


Fig. 6. Comparison of scheduling results of microturbine and exchange power.

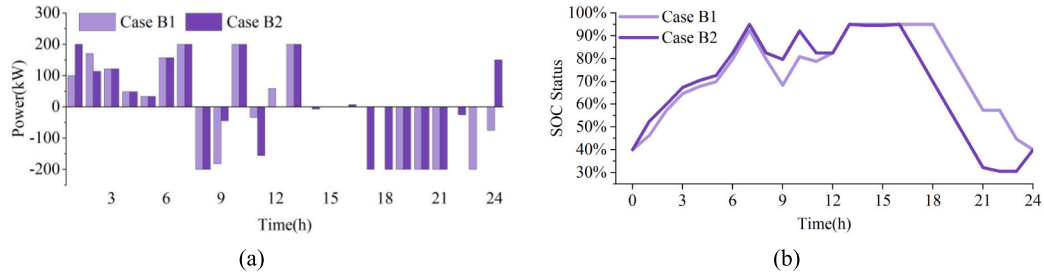


Fig. 7. Comparison of scheduling results of energy storage.

fragmented computing resources of GPU-based servers P100 and M32. Fig. 5(d) shows a significant increase in CC load from 9 h to 15 h, as many workloads are executed in fragmented computing resources. This demonstrates that the load-shifting capability of CC is fully utilized through CRDP. During this period, the CC processes more workloads at lower tariffs, improving cost-effectiveness. The microgrid dispatches the CC to consume PV through demand response, resulting in economic benefits for both the CC and the microgrid. Since the load of Case B2 is higher at 15 h, Fig. 6(a) shows that the microturbine output at 15 h is greater than in Case B1. This is due to batch workloads with longer processing times. At 15 h, the server processes both batch workloads from the backup queue and those previously allocated during dispatches, leading to higher server utilization in Case B2. The increased workload raises the CC load at 15 h. From 9 h to 15 h, Case B2 consumes more PV generation than Case B1 due to the higher CC load. As a result, the energy storage charges only at 10 h and 13 h, while discharging at 11 h and 14 h. The discharged power is greater than in Case B1 during this period.

At 16 h, the CRDP phase ends as the operational load of CC is sufficient to support the excess PV generation. Consequently, the number of backup queue workloads for Case B2 in Fig. 5(c) begins to rise at 16 h. All workloads scheduled during the CRDP phase are processed before 16 h, leading to a drop in server utilization in Fig. 5(b), and the load curve in Fig. 5(d) shows a sudden decrease at 16 h. Due to the processing of many workloads in advance, the scheduled work for 16 h to 24 h is significantly reduced. The decrease in batch workloads in the backup queue between 16 h and 24 h is evident in Fig. 5(c), where the utilization of GPU server K80 drops by 20 %. Consequently, the CC load of Case B2 is lower than that of Case B1. From 16 h to 18 h, the microgrid in both Case B1 and Case B2 are powered by a combination of PV, microturbine,

and energy storage. Between 19 h and 21 h, Case B2 sells more electricity to the distribution grid due to its smaller total load compared to Case B1 and higher sale tariffs during this period. In Case B2, all energy storage is discharged between 17 h and 21 h, so the microgrid is primarily powered by the microturbine from 22 h to 23 h. At 24 h, the energy storage of Case B2 is recharged to restore the SOC to 40 %, matching the initial dispatch state.

Overall, the scheduling method proposed in this paper effectively enhances the economic performance of microgrid operations and promotes PV consumption.

6.2.2. Effect of PV penetration on CRDP

The CC scheduling workloads considering CRDP are required to refer to the remaining unabated PV generation, meaning that changes in PV penetration also affect CC scheduling. To explore the effect of PV penetration on adaptive CRDP, this paper presents three cases for comparative analysis.

Case C1. Penetration at 40 % of PV.

Case C2. Penetration at 45 % of PV.

Case C3. Penetration at 50 % of PV.

The detailed parameters of PV generation at 40 % and 45 % penetration are given in the Appendix, while Table 5 of Appendix shows the PV generation at 50 % penetration.

To better compare how CRDP promotes PV consumption and improves microgrid economy under different penetration rates, the maximum transmission power of contact line in Cases C1 to C3 is reduced to 400 kW. The amount of discarded PV in the CC microgrid and the total operating cost for each case are shown in Table 2. As PV penetration increases, CRDP improves PV consumption, rising from 48.51 kW to 415.10 kW. Although operating costs increase in Case C1, higher PV penetration in Cases C2 and C3 reduces microgrid operating costs by 8.40 % and 8.77 %, respectively.

Figs. 8, 9, and 10 display the number of workloads in backup queues, the heat map of CC server utilization, and the CC load profile for the Case C1 and Case C2, respectively. Without CRDP, the number of backup queues, server utilization, and load remain the same across Cases C1 to C3 because the fragmentation of computing resources is identical. The following section analyzes the differences in performing CRDP under each case.

Under Cases C1 to C3, when the remaining PV power generation surpasses the normal operating load, the CC employs CRDP to consolidate idle server cores and allocate them to backup queue workloads. Higher workload processing volume elevates server utilization, increasing CC load and enhancing PV consumption efficiency. Additionally, during each period, the CC assesses whether the dispatched load exceeds the remaining PV generation. When dispatched load exceeds remaining PV generation, the CC discontinues CRDP usage during that period to limit load growth, ensuring efficient synergy between CC load and PV generation. When PV penetration rises, the daily duration where remaining PV generation surpasses normal load extends. Simultaneously, the gap between PV generation and normal load widens. Therefore, differences between the cases occur only during the CRDP period. PV penetration impacts both the duration of CRDP usage by the CC and the magnitude of CC load increase. Fig. 10 shows that the CRDP period is from 10 h to 15 h in Case C1, from 9 h to 15 h in Case C2, and from 9 h to 16 h in Case C3. The following section analyzes each case.

Table 2
Comparison of PV curtailment and operating costs.

		PV curtailment (kW)	Relative reduction in PV curtailment (kW)	Operating costs ($)	Relative cost reduction rate
Case C1	Without CRDP	143.63	–	6,127,538.42	–
	With CRDP	95.12	48.51	6,202,484.06	−1.23 %
Case C2	Without CRDP	790.36	–	5,947,042.65	–
	With CRDP	514.43	275.92	5,447,391.94	8.40 %
Case C3	Without CRDP	1819.43	–	5,948,719.49	–
	With CRDP	1404.33	415.10	5,427,813.33	8.77 %

Case C1. Fig. 8(a) shows that the number of backup queues is zero during the CRDP period, indicating that all batch workloads are processed by the servers. Comparing Figs. 9(a) and 5(b), the GPU utilization of servers K80 and P100 in Case C1 increases during the CRDP period compared to the case without CRDP, while servers P100 and V100 show relatively smaller changes. This leads to a higher CC load in Case C1 during the pooling period, as shown in Fig. 10. At 15 h, fewer workloads remain in the backup queue, reducing the total number of workloads, which lowers server utilization and decreases the CC load compared to the original. However, after 16 h, the shorter CRDP period, caused by the lower PV penetration rate, results in fewer workloads being processed. Workloads scheduled for after 16 h are not efficiently handled during the CRDP period. As a result, the CC load after 16 h shows no significant difference from the original load profile.

Case C2. The pooling time in Case C2 starts early, the number of backup queues in Fig. 8(b) is reduced to zero by 9 h, indicating that a significant number of workloads have been processed on the server since 9 h. Due to the long processing time of some workloads, which leads to more workloads being processed between 9 h and 11 h and at 14 h, Fig. 9 (b) shows higher GPU utilization in servers K80, P100, and V100 for Case C2 compared to the unpooled case and Case C1. As a result, the load in Case C2 rises more during CRDP, as shown in Fig. 10, compared to Case C1 and the scenario of without CRDP. At the end of CRDP, many backup workloads that were scheduled for after 15 h are computed between 9 h and 15 h. Fig. 8(b) shows that the number of workloads in the backup queue decreases after 15 h, and Fig. 9(b) shows a significant drop in the utilization rate of server 2 between 15 h and 24 h, leading to a reduction in the CC load.

Case C3. The pooling start time in Case C3 matches that of Case C2, but the end time is one hour later, delaying the processing of some long-running workloads. This results in the peak of simultaneous processing occurring at 15 h in Case C3, compared to Case C2. Due to the extended pooling time, Case C3 processes more workloads during the pooling period. As shown in Fig. 5(b), server utilization for servers K80, P100, V100, and M32 increases compared to other cases. This leads to a higher total load during pooling, as seen in Fig. 10, and reduces the number of workloads remaining at the end of pooling. Fig. 5(b) also shows a significant decrease in server K80 utilization after 16 h, with the load in Fig. 10 decreasing accordingly.

Overall, as PV penetration increases, the duration of CRDP extends, resulting in more workloads being processed during CRDP and fewer remaining to be handled afterward. This greatly improves the effectiveness of CC load shifting. CRDP mitigates hardware resource limitations and maximizes the scheduling potential of CC. As more PV power is generated, the CC helps dissipate excess PV generation, improving the operational efficiency of microgrid.

7. Conclusion

The heavily fragmented computing resources of CC limits workload distribution and then results in CC load incapable of coordinating with PV generation effectively. In this paper, we propose a coordinated scheduling method to consume the excess PV generation considering

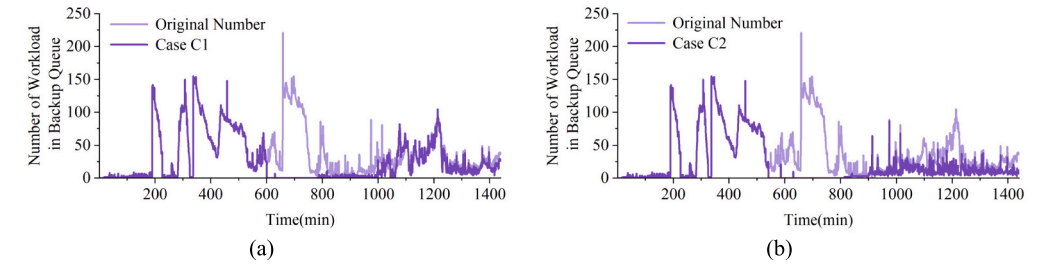


Fig. 8. Comparison of number of workloads in backup queue.

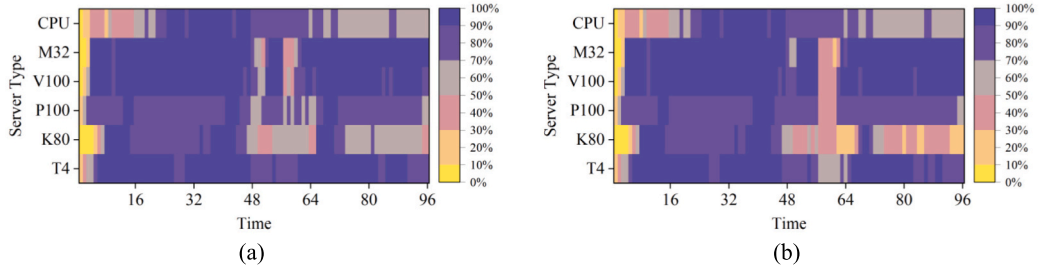


Fig. 9. Comparison of average utilization rate of various servers.

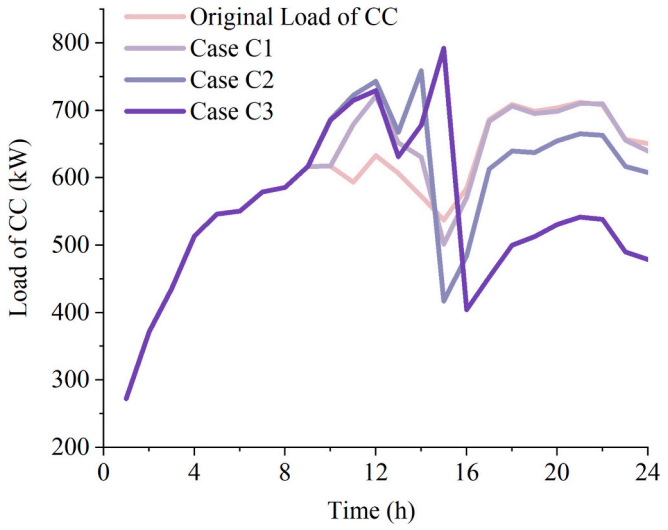


Fig. 10. Comparison of CC load in different cases.

CRDP. The workload-core mapping model is proposed to map workloads to power load based on processor core real-time states. Then a CRDP method is proposed to integrate and allocate remaining available cores according to the core real-time states. This method overcomes hardware resource limitations on load scheduling. The self-regulating CC microgrid coordinated framework is proposed to flexibly restrict the CRDP time and available cores number of computing resource pool to the volatility of PV generation. This framework achieves the balance of idle computing resources, excess PV generation and energy storage. The effectiveness of the proposed method in improving PV consumption and reducing the operational cost of CC microgrid is validated by different simulation scenarios.

CRediT authorship contribution statement

Jian Zhao: Formal analysis, Conceptualization. **Keran Huang:** Writing – original draft, Project administration, Methodology, Investigation. **Yuan Gao:** Funding acquisition, Data curation, Conceptualization. **Xiaoyan Bian:** Software, Resources. **Kai Zhang:** Supervision, Software. **Dongdong Li:** Visualization, Supervision. **Haoyang Cui:** Writing – review & editing, Visualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Supplementary data

Supplementary data to this article can be found online at https://doi.org/10.1016/j.apenergy.2025.125971.

Data availability

Server and workload data for the CC are sourced from the real production workload trace from Alibaba.

References

[1] Guo C, Luo F, Cai Z, Dong Z. Integrated energy systems of data centers and smart grids: state-of-the-art and future opportunities. Appl Energy 2021;301:117474. https://doi.org/10.1016/j.apenergy.2021.117474.
[2] Jian J, Zhao J, Ji H, Bai L, Xu J, Li P, et al. Supply restoration of data centers in flexible distribution networks with spatial-temporal regulation. IEEE Trans Smart Grid 2024;15(1):340–54. https://doi.org/10.1109/TSG.2023.3286844.
[3] Li L, Zhao J, Wang X, Xu Z, Zhu Y. Transformer-customer connectivity relationship identification for low-voltage distribution system with high penetration of household PV systems. IEEE Trans Smart Grid doi:https://doi.org/10.1109/TSG.2024.3396462.
[4] Wu S, Wang Q, Chen Q, Yu C, Tang Y. Cyber-physical integrated planning of distribution networks considering spatial-temporal flexible resources. Prot Contr Mod Pow 2024;9(3):142–56. https://doi.org/10.23919/PCMP.2023.000316.
[5] Wang K, Ye L, Yang S, Deng Z, Song J, Li Z, et al. A hierarchical dispatch strategy of hybrid energy storage system in internet data center with model predictive control. Appl Energy 2023;331:120414. https://doi.org/10.1016/j.apenergy.2022.120414.
[6] Chen M, Gao C, Shahidehpour M, Li Z, Chen S, Li D. Internet data center load modeling for demand response considering the coupling of multiple regulation methods. IEEE Trans Smart Grid 2021;12(3):2060–76. https://doi.org/10.1109/TSG.2020.3048032.
[7] Li J, Qi W. Toward optimal operation of internet data center microgrid. IEEE Trans on Smart Grid 2016;9(2):971–9. https://doi.org/10.1109/TSG.2016.2572402.

[8] Qi W, Li J, Liu Y, Liu C. Planning of distributed internet data center microgrids. IEEE Trans on Smart Grid 2017;10(1):762–71. https://doi.org/10.1109/TSG.2017.2751756.
[9] Wan T, Tao Y, Qiu J, Lai S. Internet data centers participating in electricity network transition considering carbon-oriented demand response. Appl Energy 2023;329:120305. https://doi.org/10.1016/j.apenergy.2022.120305.
[10] Liu S, Zhao T, Liu X, Li Y, Wang P. Proactive resilient day-ahead unit commitment with cloud computing data centers. IEEE Trans Ind Appl 2022;58(2):1675–84. https://doi.org/10.1109/TIA.2022.3145761.
[11] Chen M, Gao C, Shahidehpour M, Li Z. Incentive-compatible demand response for spatially coupled internet data centers in electricity markets. IEEE Trans Smart Grid 2021;12(4):3056–69. https://doi.org/10.1109/TSG.2021.3053433.
[12] Wang J, Deng H, Liu Y, Guo Z, Wang Y. Coordinated optimal scheduling of integrated energy system for data center based on computing load shifting. Energy 2023;267:126585. https://doi.org/10.1016/j.energy.2022.126585.
[13] Lian Y, Li Y, Zhao Y, Yu C, Zhao T, Wu L. Robust multi-objective optimization for islanded data center microgrid operations. Appl Energy 2023;330:120344. https://doi.org/10.1016/j.apenergy.2022.120344.
[14] Cao Y, Cheng M, Zhang S, Mao H, Wang P, Li C, et al. Data-driven flexibility assessment for internet data center towards periodic batch workloads. Appl Energy 2022;324:119665. https://doi.org/10.1016/j.apenergy.2022.119665.
[15] Ding Z, Cao Y, Xie L, Lu Y, Wang P. Integrated stochastic energy management for data center microgrid considering waste heat recovery. IEEE Trans Ind Appl 2019;55(3):2198–207. https://doi.org/10.1109/TIA.2018.2890789.
[16] Zeng Q, Du Y, Huang K, Leung K. Energy-efficient resource management for federated edge learning with CPU-GPU heterogeneous computing. IEEE Trans on Wirel Commun 2021;20(12):7947–62. https://doi.org/10.1109/TWC.2021.3088910.
[17] Rosenfeld V, Breß S, Markl V. Query processing on heterogeneous CPU/GPU systems. ACM Comput Surv 2022;55(1):1–38. https://doi.org/10.1145/3485126.
[18] Ye Z, Gao W, Hu Q, Sun P, Wang X, Luo Y, et al. Deep learning workload scheduling in GPU datacenters: a survey. ACM Comput Surv 2024;56(6):1–38. https://doi.org/10.1145/3638757.
[19] Demoulin HM, Fried J, Pedisich I, Kogias M, Loo BT, Phan LTX, et al. When idling is ideal: Optimizing tail-latency for heavy-tailed datacenter workloads with perséphone. Proc ACM SIGOPS 28th Symp Operating Syst Principles (SOSP) 2021:621–37. https://doi.org/10.1145/3477132.3483571.
[20] Iqbal W, Berral JL, Carrera D. Adaptive sliding windows for improved estimation of data center resource utilization. Future Gener Comp Sy 2020;104:212–24. https://doi.org/10.1016/j.future.2019.10.026.
[21] Zhang W, Zhang Z, Zeadally S, Chao HC, Leung VCM. Energy-efficient workload allocation and computation resource configuration in distributed cloud/edge computing systems with stochastic workloads. IEEE J on Sel Comm 2020;38(6):1118–32. https://doi.org/10.1109/JSAC.2020.2986614.
[22] Liu W, Yan Y, Sun Y, Mao H, Cheng M, Wang P, et al. Online job scheduling scheme for low-carbon data center operation: an information and energy nexus perspective. Appl Energy 2023;338:120918. https://doi.org/10.1016/j.apenergy.2023.120918.
[23] Kang Y, Pan L, Liu S. An online algorithm for scheduling big data analysis jobs in cloud environments. Knowl-Based Syst 2022;245:108628. https://doi.org/10.1016/j.knosys.2022.108628.
[24] Zhang Y, Zou B, Jin X, Luo Y, Song M, Ye Y, et al. Mitigating power grid impact from proactive data center workload shifts: a coordinated scheduling strategy integrating synergistic traffic-data-power networks. Appl Energy 2025;377:124697. https://doi.org/10.1016/j.apenergy.2024.124697.
[25] Kaur K, Garg S, Aujla GS, Kumar N, Zomaya AY. A multi-objective optimization scheme for job scheduling in sustainable cloud data centers. IEEE Trans on Cloud Comput 2019;10(1):172–86. https://doi.org/10.1109/TCC.2019.2950002.
[26] Zhao J, Chen Y, Wang X. Low voltage distribution network TN-S impedance identification considering single and three-phase hybrid connection. IEEE Trans on Ind Inform 2024;20(12):13719–30. https://doi.org/10.1109/TII.2024.3431090.
[27] da Silva NCO, Scarpin CT, Pécora Jr JE, Ruiz A. Online single machine scheduling with setup times depending on the jobs sequence. Comput Ind Eng 2019;129:251–8. https://doi.org/10.1016/j.cie.2019.01.038.
[28] Xiong H, Wang H, Shi S, Chen K. Comparison study of dispatching rules and heuristics for online scheduling of single machine scheduling problem with predicted release time jobs. Expert Syst Appl 2024;243:122752. https://doi.org/10.1016/j.eswa.2023.122752.
[29] Fan W, He J, Han Z, Li P, Wang R. Intelligent resource scheduling based on locality principle in data center networks. IEEE Commun Mag 2020;58(10):94–100. https://doi.org/10.1109/MCOM.001.1900324.
[30] Yu X, Zhao J, Zhang H, Wang X, Bian X. Data-driven distributed grid topology identification using backtracking Jacobian matrix approach. IEEE Trans on Ind Inform 2024;20(2):1711–20. https://doi.org/10.1109/TII.2023.3280936.
[31] Han H, Xu Y, Wu C, Jiang X, Cao S, Zang H, et al. Nash equilibrium-based two-stage cooperative operation strategy for multi-microgrids considering uncertainty. Prot Contr Mod Pow 2024;9(6):42–57. https://doi.org/10.23919/PCMP.2024.000295.
[32] Ma X, Xu H, Gao H, Bian M, Hussain.. Real-time virtual machine scheduling in industry IoT network: a reinforcement learning method. IEEE trans on Ind Inform 2023;19(2):2129–39. https://doi.org/10.1109/TII.2022.3211622.
[33] Ye G, Gao F, Fang J. A mission-driven two-step virtual machine commitment for energy saving of modern data centers through UPS and server coordinated optimizations. Appl Energy 2022;322:119467. https://doi.org/10.1016/j.apenergy.2022.119467.
[34] Abohamama AS, Hamouda E. A hybrid energy–aware virtual machine placement algorithm for cloud environments. Expert Syst Appl 2020;150:113306. https://doi.org/10.1016/j.eswa.2020.113306.
[35] Zhou Z, Shojafar M, Alazab M, Abawajy J, Li FM. AFED-EF: an energy-efficient VM allocation algorithm for IoT applications in a cloud data center. IEEE Trans Green Commun Netw 2021;5(2):658–69. https://doi.org/10.1109/TGCN.2021.3067309.
[36] Liu X, Cheng B, Wang S. Availability-aware and energy-efficient virtual cluster allocation based on multi-objective optimization in cloud datacenters. IEEE Trans Netw Serv 2020;17(2):972–85. https://doi.org/10.1109/TNSM.2020.2975580.
[37] Liu X, Cheng B, Wang S, Chen J. Joint availability enhancement and traffic optimization of virtual cluster allocation in cloud datacenters. IEEE Trans Netw Serv 2020;17(3):1554–67. https://doi.org/10.1109/TNSM.2020.2994146.
[38] Maldonado-Carrascosa FJ, García-Galán S, Valverde-Ibáñez M, Marciniak T, Szczerska M, Ruiz-Reyes N. Game theory-based virtual machine migration for energy sustainability in cloud data centers. Appl Energy 2024;372:123798. doi:10.1016/j.apenergy.2024.123798.